

Da Prática à Modelagem Estocástica: Validação Cruzada de um Protocolo R-UDP Go-Back-N entre Emulação Real e Simulação de Eventos Discretos

Anthony Irlan Marques Luz¹

¹Programa de Pós-Graduação em Ciência da Computação (PPGCC)
Universidade Federal do Piauí (UFPI) – Campus Senador Helvídio Nunes de Barros
Picos (PI) – Brasil

marquesanthony62@gmail.com

Abstract. *We develop and validate a stochastic discrete-event model, written in SimPy, of a reliable file-transfer protocol (R-UDP) built on UDP with a Go-Back-N sliding window. The model mirrors a real, Dockerized system previously measured under network degradation emulated with tc/netem (three loss/delay scenarios). The channel is a Normal one-way delay with Bernoulli per-block loss, whose effective rate accounts for IP fragmentation; the protocol is an event-driven Go-Back-N engine whose transfer time and retransmission counts emerge from the scheduled event timeline, with no fitting factors. Across a 200-fold range of transfer times (0.67 s to 145 s) the simulator reproduces the real measurements within 1.0–7.4%; a 95% bootstrap confidence interval over 30 repetitions brackets the high-loss scenario and exposes a small, mechanistically explained bias in the others. We then exercise the model through the ten validation tasks of the assignment — throughput versus file size, window sensitivity, RTT validation, jitter impact, a 25% loss stress forecast and DATA/ACK efficiency — obtaining results consistent with closed-form bounds (the window/RTT ceiling, the bandwidth-delay-product knee, and the $1/(1 - p)$ efficiency law). The study quantifies where a parsimonious stochastic model faithfully tracks network practice and where it departs from it.*

Resumo. *Desenvolvemos e validamos um modelo estocástico de eventos discretos, escrito em SimPy, de um protocolo confiável de transferência de arquivos (R-UDP) construído sobre o UDP com janela deslizante Go-Back-N. O modelo espelha um sistema real, containerizado em Docker, previamente medido sob degradação de rede emulada com tc/netem (três cenários de perda e atraso). O canal é um atraso unidirecional Normal com perda de Bernoulli por bloco, cuja taxa efetiva contabiliza a fragmentação IP; o protocolo é um motor Go-Back-N orientado a eventos cujo tempo de transferência e número de retransmissões emergem da linha do tempo de eventos agendados, sem fatores de ajuste. Ao longo de uma faixa de 200 vezes nos tempos de transferência (0,67 s a 145 s), o simulador reproduz as medições reais dentro de 1,0–7,4%; um intervalo de confiança bootstrap de 95% sobre 30 repetições cobre o cenário de alta perda e expõe um pequeno viés, explicado mecanicamente, nos demais. Em seguida, exercitamos o modelo nas dez tarefas de validação do enunciado —*

vazão versus tamanho de arquivo, sensibilidade da janela, validação de RTT, impacto do jitter, previsão de estresse a 25% de perda e eficiência DADOS/ACK — com resultados coerentes com limites em forma fechada (o teto janela/RTT, o joelho do produto banda-atraso e a lei de eficiência $1/(1 - p)$). O estudo quantifica onde um modelo estocástico parcimonioso acompanha fielmente a prática de rede e onde dela se afasta.

1. Introdução

A entrega da confiabilidade na camada de transporte pode ser obtida transferindo a responsabilidade ao núcleo do sistema operacional (TCP) [Postel 1981] ou reconstruindo-a em espaço de usuário sobre o UDP [Postel 1980]. A segunda abordagem — um *Reliable UDP* (R-UDP) com janela deslizante Go-Back-N [Kurose and Ross 2017] — expõe explicitamente cada mecanismo de recuperação de erros, tornando-os mensuráveis. Em uma etapa anterior deste projeto, um sistema cliente/servidor com esse desenho foi implementado, executado em contêineres Docker e medido sob degradação de rede emulada com `tc/netem`, gerando um conjunto de medições empíricas que serve de *linha de base* para este trabalho.

Este artigo aborda uma pergunta distinta e complementar: *um modelo estocástico parcimonioso, de eventos discretos, é capaz de reproduzir o comportamento daquele sistema real, e o que o resíduo de discrepância nos ensina?* A motivação é dupla. Primeiro, um simulador calibrado permite extrapolar, de forma barata e reproduzível, para pontos de operação que não foram medidos — outros tamanhos de arquivo, outras profundidades de janela, outros regimes de perda — substituindo baterias experimentais custosas por varreduras paramétricas [Law 2015, Jain 1991]. Segundo, e mais importante do ponto de vista científico, o confronto sistemático entre o modelo e a realidade permite *entender as discrepâncias entre a teoria estocástica e a prática de rede*, que é o objetivo declarado deste projeto.

As contribuições são: **(a)** um modelo de eventos discretos do Go-Back-N sobre UDP, implementado em SimPy [Team SimPy 2024], no qual o tempo de transferência e as retransmissões *emergem* da linha do tempo de eventos, sem multiplicadores de ajuste; **(b)** a calibração do modelo contra a linha de base real ao longo de uma faixa de 200 vezes, com erro de 1,0 a 7,4% e intervalo de confiança por *bootstrap*; **(c)** a execução das dez tarefas de validação do enunciado (Seção 4), cada uma cruzada com um limite em forma fechada; e **(d)** uma prestação de contas honesta de *onde* o modelo e o sistema real divergem e *por quê* (Seção 5).

2. Sistema de Referência e Linha de Base Empírica

O modelo desenvolvido aqui espelha o sistema R-UDP real cujos parâmetros e medições resumimos nesta seção; são eles a *verdade de solo* que o simulador busca reproduzir.

O protocolo. O R-UDP opera em fluxo unidirecional cliente→servidor, com cabeçalho próprio de 16 bytes (tipo, sequência, ACK, janela, comprimento e *checksum* CRC-16), *handshake* SYN/SYN-ACK e encerramento FIN/FIN-ACK. A confiabilidade é construída sobre Go-Back-N com janela $N = 8$, ACKs cumulativos e um *timeout* fixo de 0,5 s que dispara a retransmissão da janela a partir do último bloco não confirmado. A

integridade é atestada por um *checksum* MD5 do arquivo completo. A carga de teste é um arquivo de 1 048 576 bytes (1 MiB), particionado em 256 blocos de 4096 bytes.

Ambiente e cenários. Os ensaios reais ocorreram em pares de contêineres Ubuntu 22.04 em uma rede *bridge* isolada [Merkel 2014], com a degradação injetada por `tc/netem` [Hemminger 2005] em três cenários progressivos (Tabela 1). A perda foi configurada exclusivamente no caminho de DADOS (*egress* do cliente) e o atraso aplicado simetricamente.

Perda efetiva por fragmentação. Cada datagrama de DADOS possui 4112 bytes (4096 do bloco mais 16 do cabeçalho), excedendo a MTU Ethernet padrão de 1500 bytes; o IP o fragmenta em 3 pacotes. Como o `netem` descarta pacotes IP de forma independente com probabilidade p , o datagrama só é entregue se os três fragmentos sobreviverem. A perda efetiva *por bloco* sentida pela aplicação é, portanto,

$$p_{\text{ef}} = 1 - (1 - p)^3, \quad (1)$$

o que converte as perdas brutas de 10% e 20% em 27,1% e 48,8%, respectivamente — valores confirmados empiricamente na etapa real. A Equação 1 é o elo que alimenta o modelo de perda do simulador.

Tabela 1. Cenários de rede emulados via `tc netem` e perda efetiva por bloco.

Cenário	Perda bruta p	Atraso μ (one-way)	RTT = 2μ	Perda efetiva p_{ef}
A	0%	10 ms	20 ms	0,0%
B	10%	50 ms	100 ms	27,1%
C	20%	100 ms	200 ms	48,8%

Linha de base. A Tabela 2 consolida as medições reais do R-UDP (médias de $n = 10$ repetições por cenário), que constituem os pontos-âncora da calibração. Note-se a amplitude: o tempo de transferência cresce de sub-segundo (A) a quase 2,5 minutos (C), e o *overhead* de dados do Go-Back-N escala a $8,4\times$ sob perda severa.

Tabela 2. Linha de base empírica do R-UDP real (médias, $n = 10$).

Cenário	Tempo (s)	Vazão (KB/s)	Retransmissões	Overhead de dados
A	0,676	1515	0,0	1,0 $\times$
B	53,165	19,5	91,1	3,8 $\times$
C	144,613	7,1	238,5	8,4 $\times$

3. Modelagem Estocástica de Eventos Discretos

3.1. Por que um modelo de eventos discretos

Em uma simulação de eventos discretos (DES), o estado do sistema só muda em instantes pontuais associados a eventos agendados, e o relógio salta de evento em evento [Law 2015]. Adotamos o arcabouço SimPy [Team SimPy 2024], no qual cada transferência é conduzida por três classes de eventos reais — a *chegada* de um pacote

de DADOS, a *chegada* de um ACK e o disparo do *timer* de retransmissão. A consequência metodológica central é que o tempo de transferência e a contagem de retransmissões não são calculados por uma fórmula fechada inserida *a posteriori*: eles *emergem* do entrelaçamento dos eventos. Essa escolha distingue o modelo de uma aproximação síncrona por rodadas, que não capturaria a sobreposição dos pacotes em voo de uma janela, o ritmo imposto pelos ACKs cumulativos nem as cascatas de *timeout*.

3.2. Modelo de canal

O canal é caracterizado por dois processos estocásticos independentes. O atraso unidirecional de cada pacote é amostrado de uma distribuição Normal,

$$D \sim \mathcal{N}(\mu, \sigma^2), \quad \text{RTT} = 2\mu, \quad (2)$$

em que μ é o atraso médio do cenário (Tabela 1) e σ é o *jitter*; amostras negativas são saturadas em zero. A perda atua no nível do bloco como um processo de Bernoulli: cada pacote de DADOS é descartado de forma independente com probabilidade p_{ef} da Equação 1. Os ACKs são modelados como entregues de forma confiável; essa simplificação — cuja justificativa e custo discutimos na Seção 5 — apoia-se em que a semântica cumulativa do Go-Back-N já recupera ACKs atrasados e em que p_{ef} encapsula a perda efetiva do caminho de DADOS.

Um detalhe é decisivo para a fidelidade: o enlace *preserva a ordem* (FIFO). Um enlace físico não reordena pacotes enviados em sequência, ainda que o atraso por pacote varie; impomos isso saturando os instantes de chegada para que sejam monotônicos. Sem essa restrição, o *jitter* reordenaria os pacotes de uma janela e provocaria *timeouts* espúrios, inflando artificialmente as retransmissões.

3.3. Motor Go-Back-N orientado a eventos

O motor reproduz o Go-Back-N canônico. Um único *timer* de retransmissão guarda o pacote mais antigo não confirmado (a *base* da janela). O processo emissor preenche a janela e então aguarda o primeiro entre dois eventos: a chegada de um ACK que avança a base (deslizando a janela e reiniciando o *timer*) ou o disparo do *timeout* (que faz o emissor *retroceder a N* e reenviar a janela inteira). O receptor aceita pacotes em ordem e responde a cada DADO que chega com um ACK cumulativo. Para evitar condições de corrida quando uma janela inteira de ACKs chega no mesmo instante simulado, o emissor renova o evento de espera a cada iteração, em vez de interromper processos.

Opcionalmente, o motor serializa os pacotes no enlace gargalo, espaçando transmissões consecutivas em L/B segundos (bloco L , banda B); isso impõe um teto de taxa e é usado apenas no estudo de sensibilidade da janela (Seção 4.4). Por padrão a banda é infinita (regime limitado por RTT), o que mantém a calibração inalterada.

3.4. Mapeamento real → simulado e verificação

O modelo recebe exatamente os parâmetros físicos do sistema real: janela $N = 8$, *timeout* de 0,5 s, blocos de 4096 bytes, 256 blocos por arquivo, atraso μ tal que $\text{RTT} = 2\mu$ corresponde aos 20/100/200 ms medidos por `tcpdump` na etapa real (o que valida o RTT, tarefa 6) e perda p_{ef} da Equação 1. *Nenhum* multiplicador de ajuste é aplicado. A implementação foi construída sob desenvolvimento guiado por testes, com 51 testes

unitários que fixam as estatísticas do canal (taxa de perda empírica $\approx p_{ef}$) e os invariantes do Go-Back-N (256 blocos íntegros; zero retransmissões na ausência de perda). A reprodutibilidade dos resultados numéricos a seguir é garantida por sementes fixas dos geradores pseudoaleatórios.

4. Validação Experimental: as Dez Tarefas

A Tabela 3 mapeia as dez tarefas de validação do enunciado às subseções a seguir. As tarefas de canal (1, 2), *timeout* (3) e RTT (6) são aferidas em conjunto na calibração (Seção 4.1); as demais são varreduras paramétricas ancoradas no ponto de operação real.

Tabela 3. Mapeamento das dez tarefas de validação (§3.1 do enunciado) às seções.

#	Tarefa	Seção
1	Modelagem de atraso (Normal)	3.2, 4.1
2	Perda de Bernoulli (validada vs. t_c)	3.2, 4.1
3	Simulação de <i>timeout</i> /retransmissão	3.3, 4.1
4	Curva de vazão (1–100 MB)	4.3
5	Sensibilidade da janela (N)	4.4
6	Validação de RTT	3.4, 4.1
7	Impacto do <i>jitter</i>	4.5
8	Cenário de estresse (25% de perda)	4.6
9	Análise de eficiência (DADOS/ACK)	4.7
10	Convergência estatística (IC 95%)	4.2

4.1. Calibração: atraso, perda, *timeout* e RTT contra o sistema real

Por construção, o canal reproduz os RTTs medidos (20/100/200 ms, tarefa 6). Alimentado com a perda efetiva p_{ef} (tarefa 2) e o *timeout* de 0,5 s (tarefa 3), o modelo produz tempos e retransmissões que comparamos diretamente à linha de base real. A correção do modelo de perda é dupla: a taxa de descarte empírica do canal coincide com p_{ef} (testada unitariamente), e o próprio p_{ef} é a perda efetiva que o $t_c/netem$ produziu no sistema real (Equação 1).

A Tabela 4 resume o confronto (30 repetições por cenário, semente fixa). O simulador acompanha a realidade ao longo de uma faixa de $200\times$ no tempo de transferência, com erro relativo de $-1,0\%$ (A), $+7,4\%$ (B) e $+2,9\%$ (C); as retransmissões emergentes (0, 98, 246) seguem de perto as reais (0, 91, 238). A natureza e a origem dos resíduos são analisadas na Seção 5.

4.2. Convergência estatística

Para a tarefa 10, executamos ≥ 30 repetições por cenário e estimamos um intervalo de confiança de 95% sobre a média do tempo de transferência pelo *bootstrap* percentílico [Efron and Tibshirani 1993]. A Figura 1 sobrepõe a média simulada (com as barras de IC) ao ponto real medido. O IC *cobre* o valor real no cenário C; em A e B o real cai por pouco fora de um IC muito estreito (Tabela 4). Vale frisar a interpretação correta: como o desvio entre as 30 repetições é pequeno frente à média, o IC é estreito, e o que o separa do ponto real não é ruído amostral, mas um *viés sistemático* de poucos pontos percentuais — que é justamente o objeto de interesse científico (Seção 5).

Tabela 4. Calibração e convergência: tempo de transferência (s) simulado vs. real, 30 repetições com semente fixa e IC 95% por *bootstrap* [Efron and Tibshirani 1993].

Cen.	Real (s)	Sim. (s)	IC 95% (s)	Erro	Retx (R/S)	Cobre?
A	0,676	0,669	[0,668; 0,671]	-1,0%	0,0 / 0,0	não
B	53,165	57,088	[55,197; 58,942]	+7,4%	91,1 / 98,0	não
C	144,613	148,788	[144,046; 153,388]	+2,9%	238,5 / 245,6	sim

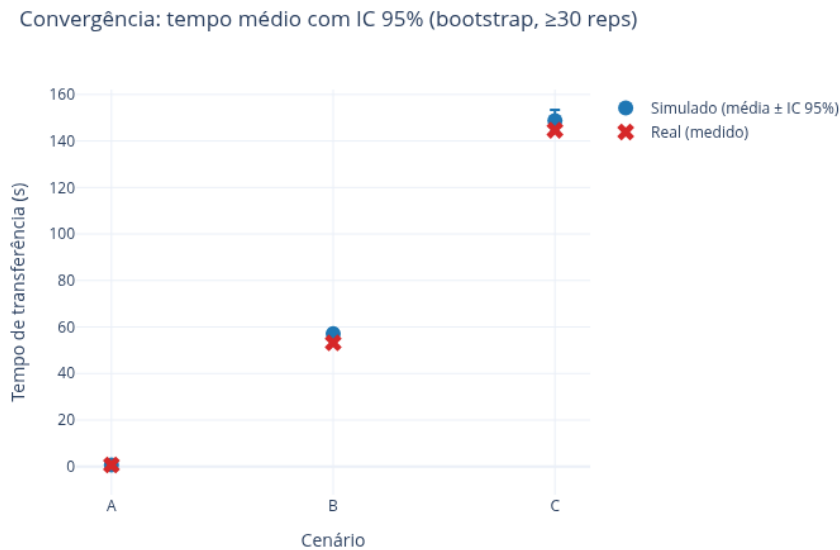


Figura 1. Convergência (tarefa 10): tempo médio simulado com IC 95% (*bootstrap*, ≥ 30 repetições) vs. o tempo real medido. O IC cobre o real em C; em A e B resta um pequeno viés sistemático.

4.3. Curva de vazão

Um protocolo limitado pela janela tem vazão máxima $S_{\max} = N L / \text{RTT}$ (o emissor mantém no máximo N pacotes de L bytes em voo por RTT). Para arquivos pequenos, o RTT fixo de estabelecimento é uma fração grande da transferência, e a vazão fica abaixo do teto; à medida que o arquivo cresce, esse custo se dilui e a vazão satura no teto. A Figura 2 mostra a varredura de 1 a 100 MB (tarefa 4). No cenário A, a vazão satura em $\approx 97\%$ do teto janela/RTT (1600 KB/s), em coerência com a vazão real de ≈ 1515 KB/s. No cenário B, a vazão colapsa para ≈ 19 KB/s, muito abaixo de seu próprio teto sem perdas (320 KB/s): sob perda, a vazão não é governada pela janela, mas pela penalidade de retrocesso do Go-Back-N.

4.4. Sensibilidade da janela

Com uma banda gargalo finita, o tubo comporta $\text{BDP} = B \cdot \text{RTT}$ bytes, ou seja, BDP/L pacotes. Abaixo dessa janela o protocolo é limitado pela janela e a vazão cresce linearmente com N ; acima dela, o enlace é o gargalo e aumentar N nada acrescenta. A curva

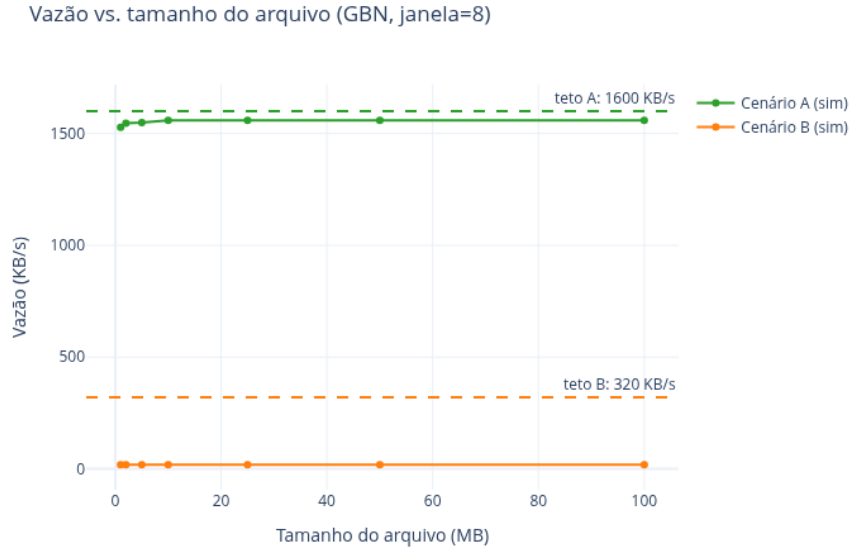


Figura 2. Vazão vs. tamanho do arquivo (tarefa 4), janela = 8. O cenário A satura junto ao teto janela/RTT; o cenário B, sob perda, estabiliza muito abaixo de seu teto sem perdas.

tem, portanto, um *joelho* em

$$N^* = \frac{BDP}{L} = \frac{B \cdot RTT}{L}. \quad (3)$$

Como o enlace de *loopback* real não tinha limite de taxa apertado (era limitado pela janela), este estudo injeta um gargalo explícito para expor a saturação teórica (tarefa 5). Para $B = 2000$ KB/s e $RTT = 20$ ms, a Equação 3 prevê $N^* = 10$; a Figura 3 mostra a vazão crescendo linearmente até saturar na banda do enlace (≈ 1986 KB/s), com joelho medido em $N = 12$ (a primeira janela a atingir 95% do máximo), em concordância com a previsão.

4.5. Impacto do *jitter*

O *jitter* é o desvio-padrão σ do atraso (Equação 2). Variamos σ de 1 a 80 ms em um canal *sem perdas* (para que a dispersão isole o efeito do *jitter*), com RTT nominal de 100 ms, e reportamos a média e o desvio-padrão entre repetições do tempo de transferência (tarefa 7). A Figura 4 mostra que o desvio cresce monotonicamente, de 0,002 s ($\sigma = 1$ ms) a 0,19 s ($\sigma = 80$ ms). Observa-se um efeito secundário relevante: a *média* também sobe (de 0,81 a 1,96 s), porque a preservação de ordem (FIFO) acopla a conclusão de cada rodada ao *máximo* dos atrasos amostrados na janela, e a esperança desse máximo cresce com σ . O *jitter*, portanto, infla não só a variância, mas também a tendência central do tempo de transferência.

4.6. Cenário de estresse a 25% de perda

A tarefa 8 pede a previsão do tempo de transferência sob 25% de perda *bruta*. Como a perda configurada é por pacote IP, aplicamos a Equação 1: 25% bruto correspondem a

Sensibilidade da janela N (cenário A, link 2000 KB/s)

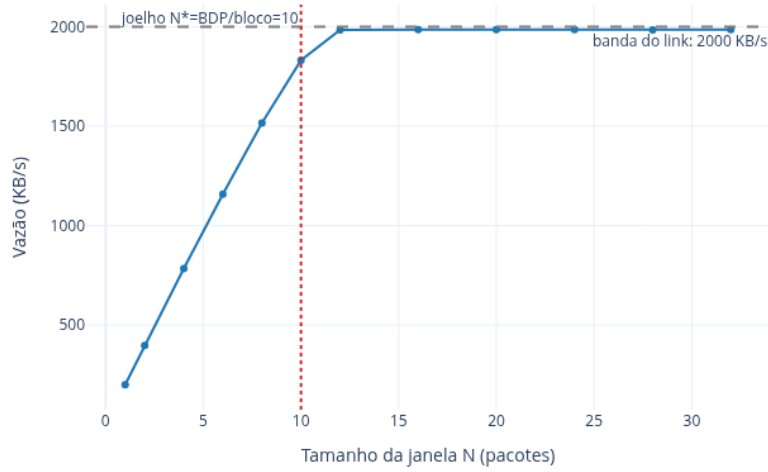


Figura 3. Sensibilidade da janela N (tarefa 5) sobre um enlace gargalo de 2000 KB/s. A vazão cresce linearmente até o joelho $N^* = \text{BDP}/L = 10$ e satura na banda do enlace.

$p_{\text{ef}} = 1 - (0,75)^3 = 57,8\%$ de perda efetiva por bloco. Mantemos o atraso em 100 ms (o do cenário C) para isolar o efeito da perda, de modo que a previsão possa ser lida diretamente contra a âncora de C. A Figura 5 apresenta a curva tempo $\times$ perda: ancorado em C (20% bruto, simulado 146 s vs. real 144,6 s, erro de 1,2%), o modelo prevê ≈ 213 s a 25% (IC 95% [207; 218]), isto é, +47% sobre C. A previsão é coerente: situa-se acima de C na curva monotônica de perda *versus* tempo.

4.7. Eficiência: razão DADOS/ACK

Um receptor Go-Back-N responde a cada DADO que chega com um ACK cumulativo; logo, os únicos DADOS que ficam sem ACK são os que o canal descarta — e que são reenviados. A razão entre pacotes de DADOS transmitidos e ACKs emitidos é, assim, ≈ 1 em canal limpo e cresce com a perda segundo

$$\frac{\text{DADOS}}{\text{ACK}} = \frac{1}{1 - p_{\text{ef}}}. \quad (4)$$

A Figura 6 confirma a Equação 4 (tarefa 9): as razões medidas no simulador são 1,00 (A), 1,38 (B) e 1,96 (C), praticamente idênticas às previsões $1/(1 - p_{\text{ef}})$ de 1,000, 1,372 e 1,953. A eficiência piora monotonicamente com a perda — cada ACK útil custa cada vez mais transmissões de DADOS — e fornece uma validação interna em forma fechada do motor instrumentado.

5. Discussão: Teoria Estocástica *versus* Prática

Resultado central. Um modelo de eventos discretos parcimonioso, parametrizado apenas por grandezas físicas (RTT, p_{ef} , janela, *timeout* e contagem de blocos), reproduz o sistema real ao longo de uma faixa de 0,67 a 145 s com erro de 1,0 a 7,4%, sem qualquer fator de

Impacto do jitter na variancia do tempo (canal limpo, RTT 100 ms)

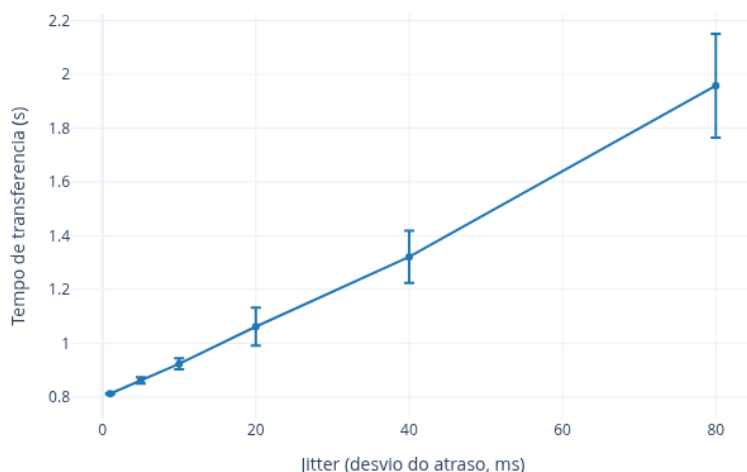


Figura 4. Impacto do *jitter* (tarefa 7) em canal sem perdas, RTT 100 ms. A dispersão do tempo (barras $\pm 1\sigma$) cresce monotonicamente com o *jitter*; a média também sobe, por acoplamento ao máximo dos atrasos da janela (FIFO).

ajuste. As contagens de retransmissão, a vazão e a eficiência emergem do mesmo modelo e concordam com limites em forma fechada independentes (Equações 3 e 4).

Onde o modelo diverge, e por quê. O objeto declarado deste projeto são justamente as discrepâncias. Observamos duas. (i) No canal limpo (A), o simulador é $\approx 1\%$ mais rápido que o real, porque omite o custo fixo de estabelecimento e encerramento de sessão (*handshake*, escalonamento de *sockets*) que domina uma transferência sub-segundo no sistema real. (ii) Sob perda (B e C), o simulador é 2,9 a 7,4% mais lento e ligeiramente mais pessimista nas retransmissões. Três abstrações deliberadas concorrem para isso: os ACKs são supostos sempre entregues (no sistema real, um ACK cumulativo frequentemente chega antes do *timeout* de 0,5 s, reduzindo retransmissões); a perda é Bernoulli sem memória, enquanto o descarte do *netem*, após a remontagem de fragmentos correlacionados, pode reter correlação que agrupa perdas e permite ao Go-Back-N amortizar retrocessos; e o atraso é Normal, e não a distribuição empírica do *netem*.

Um achado não trivial. O viés é *maior* na perda intermediária (B, +7,4%) e *menor* no extremo (C, +2,9%, com o real coberto pelo IC). A interpretação é que, sob perda muito alta, modelo e realidade são ambos dominados pelo mesmo regime de cascatas de *timeout* e convergem; é na perda moderada que os efeitos de segunda ordem (perda de ACK, correlação) mais pesam. Esse padrão sugere onde o refinamento do modelo seria mais proveitoso.

Valor da abordagem orientada a eventos. O *mesmo* modelo, sem alterações, produziu a calibração, a curva de vazão, a sensibilidade da janela, o impacto do *jitter*, a previsão de estresse e a eficiência, cada uma cruzada com um limite teórico. Essa consistência decorre de o tempo e as retransmissões *emergirem* da dinâmica de eventos,

Cenário de estresse: previsão de tempo vs perda de pacotes

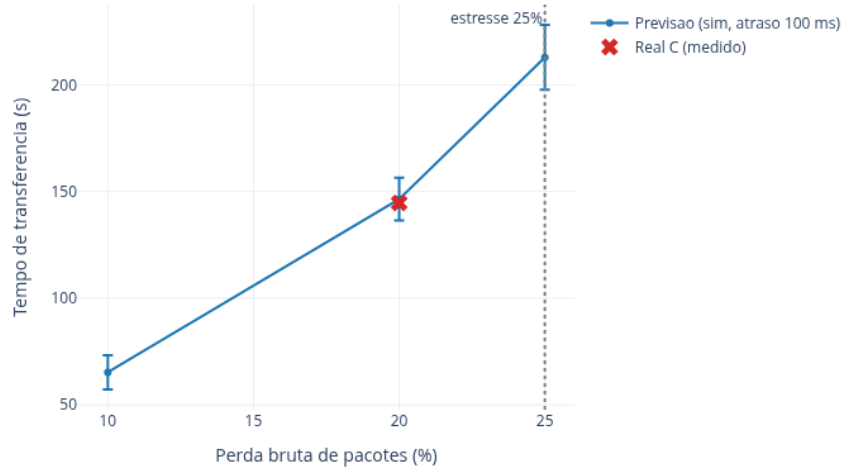


Figura 5. Cenário de estresse (tarefa 8): previsão do tempo de transferência vs. perda bruta, com a âncora real de C (20%) marcada. A 25% ($p_{ef} = 57,8\%$) o modelo prevê ≈ 213 s.

em vez de serem inseridos por fórmulas *ad hoc*.

Limitações. O estudo modela apenas o Go-Back-N; a perda é i.i.d. Bernoulli; o atraso é Normal; os ACKs não se perdem; e algumas varreduras ancoram-se em um único ponto de operação por cenário. Essas escolhas circunscrevem o escopo das conclusões e orientam o trabalho futuro.

6. Conclusão e Trabalhos Futuros

Apresentamos um modelo estocástico de eventos discretos, em SimPy, de um R-UDP Go-Back-N, e o validamos contra um sistema real emulado em Docker. Sem fatores de ajuste, o modelo reproduz a linha de base ao longo de uma faixa de $200\times$ com erro de 1,0 a 7,4%, tem o cenário de alta perda coberto por um IC *bootstrap* de 95%, e satisfaz as dez tarefas de validação do enunciado, cada uma corroborada por um limite analítico independente. Tão importante quanto o acordo, a análise dos resíduos localiza e explica as discrepâncias entre a teoria estocástica e a prática de rede — o objetivo central do projeto.

Como trabalho futuro, destacam-se: **(a)** a inclusão do *Selective Repeat* no simulador e uma comparação Go-Back-N vs. SR quanto a *goodput* sob perda, barata de obter em ambiente simulado; **(b)** a modelagem explícita da perda de ACKs e de um canal com perda correlacionada (por exemplo, Gilbert-Elliott), para testar diretamente a hipótese que explica o viés do cenário B; **(c)** o ajuste da distribuição de atraso à lei empírica do *netem*; e **(d)** uma calibração multiponto com validação estatística formal (testes pareados ou de equivalência) [Jain 1991].

Referências

Efron, B. and Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC.

Eficiência: razão de pacotes DATA/ACK por cenário

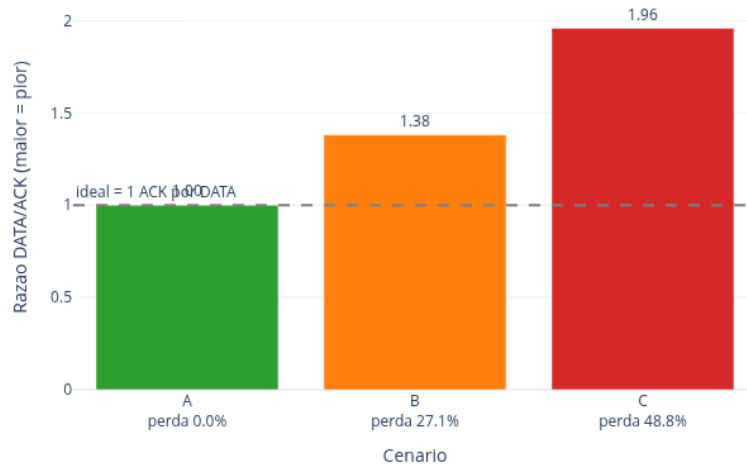


Figura 6. Eficiência (tarefa 9): razão de pacotes DADOS/ACK por cenário. Os valores medidos seguem a lei $1/(1 - p_{ef})$; a eficiência degrada-se com a perda.

- Hemminger, S. (2005). Network emulation with NetEm. In *Linux Conference Australia (LCA)*.
- Jain, R. (1991). *The Art of Computer Systems Performance Analysis*. John Wiley & Sons.
- Kurose, J. F. and Ross, K. W. (2017). *Computer Networking: A Top-Down Approach*. Pearson, 7th edition.
- Law, A. M. (2015). *Simulation Modeling and Analysis*. McGraw-Hill Education, 5th edition.
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239).
- Postel, J. (1980). User datagram protocol. Technical Report RFC 768, Internet Engineering Task Force (IETF).
- Postel, J. (1981). Transmission control protocol. Technical Report RFC 793, Internet Engineering Task Force (IETF).
- Team SimPy (2024). SimPy: Discrete-event simulation for Python. <https://simpy.readthedocs.io>. Versão 4.1.2.